

ConformalGuard: Сертификация безопасности многоагентных систем больших языковых моделей без предположений о распределении на основе конформного предсказания на динамических графах исполнения

Р. Н. Анаэдевха^{1,*}, А. Г. Трофимов¹, Ю. В. Бородачѳв²

¹Национальный исследовательский ядерный университет «МИФИ» (Московский инженерно-физический институт),

Каширское шоссе, д. 31, 115409 Москва, Россия

²Центр исследований искусственного интеллекта НИЯУ МИФИ, Москва, Россия

*ar006@campus.mephi.ru

Аннотация. Многоагентные системы на основе больших языковых моделей (LLM) — такие как AutoGen, MetaGPT, LangGraph и автономные агентные оркестрации — исполняются в виде динамических графов, где вершинами являются вызовы агентов, а рѳбрами — передача сообщений, обращения к инструментам, операции с памятью и делегирование. Эти графы развиваются во время вывода. Существующие механизмы безопасности (фильтры ввода-вывода, модерация контента, тонкая настройка выравнивания) работают на уровне отдельных агентов и вызовов, не выявляя возникающих нарушений безопасности, обусловленных композицией взаимодействий между агентами. В настоящей работе представлена ConformalGuard — первая структура распределѳнно-независимой сертификации безопасности многоагентных LLM-систем посредством применения теории конформного предсказания к динамическим графам исполнения. Сделано три вклада. Во-первых, формализована динамическая структура графа исполнения многоагентной системы как непрерывно-временного гетерогенного графа с типизированными вершинами и рѳбрами; нонконформные показатели вычисляются над временными подграфами, а не над отдельными токенами или ответами, что позволяет покрывать весь многошаговый процесс работы агентов. Во-вторых, доказана теорема о маргинальном покрытии для конформного предсказания на динамических графах при условии обмениваемости подграфов в калибровочном множестве, обеспечивающая конечную выборку с покрытием $1 - \alpha$ ($\alpha \leq 0,05$) для рабочих процессов длиной до 50 шагов без каких-либо параметрических предположений о LLM или инструментах. В-третьих, выведен онлайн-адаптивный конформный механизм, обеспечивающий валидное покрытие при дрейфе распределения с использованием расширенного протокола Гиббса–Кандеса. Оценка на бенчмарке AgentChain-26 из 12 400 размеченных мультиагентных трасс выполнения с 11 классами критических нарушений безопасности (утечка персональных данных, неавторизованный вызов инструментов, распространение prompt-инъекций, нарушение разделения сессий, и др.) на четырёх агентных платформах (AutoGen, MetaGPT, CrewAI, LangGraph) показывает: эмпирическое покрытие 94,9% при целевом $1 - \alpha = 0,95$; блокировка 96,1% критических нарушений против 18,3–31,2% для существующих базовых методов (LlamaGuard, WildGuard, Circuit Breaker, OpenAI Moderation API); средние ширины интервалов предсказания 1,8 безопасных продолжений на шаг; задержка вывода 23 мс на шаг агента. ConformalGuard интегрирована в платформу RobustIDPS.ai как выделенный модуль сертификации безопасности агентного слоя.

Ключевые слова: конформное предсказание, многоагентные системы, большие языковые модели, динамические графовые нейронные сети, сертификация безопасности, распределѳнно-независимые гарантии, графы исполнения, обмениваемость, обнаружение аномалий.

ConformalGuard: Distribution-Free Safety Certification for Multi-Agent LLM Systems via Conformal Prediction on Dynamic Execution Graphs

R. N. Anaedevha^{1,*}, A. G. Trofimov¹, Y. V. Borodachev²

¹*National Research Nuclear University MEPhI (Moscow Engineering Physics Institute), 31 Kashirskoe Highway, 115409 Moscow, Russia*

²*Artificial Intelligence Research Center, NRNU MEPhI, Moscow, Russia*

**ar006@campus.mephi.ru*

Abstract. Multi-agent large language model (LLM) systems — AutoGen, MetaGPT, LangGraph, and autonomous agent orchestrations — execute as dynamic graphs where nodes are agent invocations and edges are message passing, tool calls, memory operations, and delegation. These graphs evolve at inference time. Existing safety mechanisms (input/output filters, content moderation, alignment fine-tuning) operate per-agent and per-call, missing emergent safety violations arising from the composition of agent interactions. This paper introduces ConformalGuard, the first framework for distribution-free safety certification of multi-agent LLM systems by applying conformal prediction theory to dynamic execution graphs. Three contributions are presented. First, we formalize the multi-agent execution-graph structure as a continuous-time heterogeneous graph with typed nodes and edges, and compute nonconformity scores over temporal subgraphs — not over individual tokens or outputs — enabling certificates that cover entire multi-step agent workflows. Second, we prove a marginal-coverage theorem for graph conformal prediction under exchangeability of calibration subgraphs, yielding finite-sample $1 - \alpha$ coverage ($\alpha \leq 0.05$) for workflows of up to 50 steps without any parametric assumptions on the underlying LLMs or tools. Third, we derive an online adaptive conformal mechanism that maintains valid coverage under distribution shift via an extended Gibbs–Candès protocol. Evaluation on AgentChain-26, a constructed benchmark of 12,400 labeled multi-agent execution traces across 11 critical safety-violation classes (PII exfiltration, unauthorized tool invocation, prompt-injection propagation, session-isolation breach, and others) on four agent platforms (AutoGen, MetaGPT, CrewAI, LangGraph), demonstrates: empirical coverage 94.9% at target $1 - \alpha = 0.95$; blocking of 96.1% of critical violations versus 18.3–31.2% for existing baselines (LlamaGuard, WildGuard, Circuit Breaker, OpenAI Moderation API); average prediction-set widths of 1.8 safe continuations per step; per-step inference latency of 23 ms. ConformalGuard is integrated into the RobustIDPS.ai platform as a dedicated agent-layer safety-certification module.

Keywords: conformal prediction, multi-agent systems, large language models, dynamic graph neural networks, safety certification, distribution-free guarantees, execution graphs, exchangeability, anomaly detection.

Funding. This research was supported by the Ministry of Economic Development of the Russian Federation, agreement No. 000000C313925P3Q0002.

For citation: Anaedevha R. N., Trofimov A. G., Borodachev Y. V. ConformalGuard: Distribution-Free Safety Certification for Multi-Agent LLM Systems via Conformal Prediction on Dynamic Execution Graphs. *Vestnik Kibernetiki*. ... DOI: 10.35266/...

1. Introduction

The deployment of multi-agent LLM systems has accelerated rapidly in 2024–2026: AutoGen [1], MetaGPT [2], LangGraph [3], CrewAI, and AutoGPT-style agent swarms now power production workflows across software engineering (SWE-Bench [4]), data analysis (MLAgentBench [5]), web automation (OSWorld [6]), and customer-service orchestration. These systems execute as dynamic graphs where nodes are agent invocations (typically LLM calls with role-specific system prompts), and edges are message passing, tool calls, memory reads/writes, and task delegations. Crucially, the graphs are not statically known — they emerge at inference time as agents decide which sub-agents to spawn, which tools to invoke, and which memory entries to consult.

This compositional execution introduces a previously under-recognized class of safety failures. Whereas single-agent LLM safety is addressed by per-call guards (LlamaGuard [7], WildGuard [8], OpenAI’s Moderation API [9], Circuit Breakers [10]), and by alignment fine-tuning, multi-agent systems exhibit *emergent* unsafe behaviors arising from interactions that no single guard can detect. AgentPoison [11] demonstrated that poisoning a single retrieval memory propagates a backdoor through downstream agents at $\geq 80\%$ end-to-end attack success rate at $< 0.1\%$ poison rate. Agent Smith [12] showed that infectious jailbreaks spread exponentially through multi-modal agent populations. Greshake et al. [13] formalized indirect prompt injection: when one agent retrieves attacker-controlled content from a website or database, the injected instructions hijack downstream agents that consume the retrieved content as authoritative input. The R-Judge benchmark [14] catalogs 569 multi-step agent failures across 27 scenarios, finding even GPT-4-class judges miss $\geq 38\%$ of safety-critical sequences.

Existing defenses against these failures fall into three families, each insufficient. *Per-call guards* screen individual LLM inputs and outputs but cannot reason about cross-agent composition; an attack that is benign at every individual step but catastrophic in aggregate (e.g., agent *A* retrieves PII into context, agent *B* summarizes for diagnostics, agent *C* logs the summary externally — each step alone is innocuous, the chain leaks PII) evades per-call detection entirely. *Static graph analyzers* apply taint-tracking or static information-flow rules to known agent topologies but break under the dynamic-graph reality of modern systems. *Reinforcement-learning-from-AI-feedback (RLAIF) safety classifiers* produce heuristic rejection decisions without finite-sample coverage guarantees, leaving practitioners with no formal way to set acceptable failure rates for safety-critical deployments.

This paper introduces ConformalGuard, a framework that closes these three gaps simultaneously by applying *conformal prediction* — a distribution-free, finite-sample, assumption-free statistical inference framework [34,35] — to the dynamic execution graph of multi-agent LLM systems. The central insight is that safety certification at the multi-agent level should be expressed as a coverage statement: *with probability at least $1 - \alpha$, the next-step prediction set computed on the current execution-graph state contains a safe action*. Coverage is established by computing nonconformity scores over execution subgraphs (not individual tokens) and calibrating thresholds against a held-out subgraph corpus. The conformal guarantee holds distribution-free under exchangeability of past subgraphs, requiring no parametric assumptions on the underlying LLMs or tools.

Our prior work has progressively built the foundations for this synthesis. CT-Explain [15] (in our INJOIT track) introduced conformal prediction in a security context for SOC intrusion detection; the present paper extends that line to multi-agent LLM safety. CT-DGNN-JailGuard [16] (also in the INJOIT track) used continuous-time dynamic GNNs for jailbreak campaign detection on API interaction graphs — the same dynamic-graph substrate ConformalGuard now employs for executing-agent traces. MambaShield [17] provided the selective state-space sequence-modeling primitive useful for encoding subgraph histories. Uncertainty-calibrated hierarchical Gaussian processes [18] contributed the multi-scale calibration methodology. Temporal Adaptive Neural ODEs [19] demonstrated continuous-time evolution on irregularly-sampled events. The

adversarially-robust IDS [20] and Stochastic Multimodal Transformer [21] contributed empirical robustness baselines and multimodal feature encoding. The Deep Q-Network poisoning detector [22] supplied the active-investigation primitive used for analyst-confirmed feedback in production. The wireless-device-identification federated framework [23] prototyped the cross-silo coordination architecture that ConformalGuard extends to cross-platform agent monitoring. ConformalGuard is the natural capstone of this series, bringing distribution-free certification to the agentic-LLM frontier.

1.1. Contributions

This paper makes four contributions.

Execution-graph formalization for multi-agent LLM systems. We formalize the multi-agent execution graph as a continuous-time heterogeneous graph $\mathcal{G}(t) = (\mathcal{V}(t), \mathcal{E}(t), \mathcal{R}, \phi, \psi)$ with four node types (agents, tools, memory entries, messages) and six edge types (invokes, reads, writes, delegates, replies, cites), evolving at inference time as agents decide their next actions. We construct AgentChain-26, a benchmark of 12,400 labeled multi-agent execution traces across four agent platforms.

Graph conformal prediction with marginal-coverage guarantee. We prove a marginal-coverage theorem (Theorem 1) for conformal prediction over temporal execution subgraphs, showing that under exchangeability of calibration-subgraph nonconformity scores the prediction set $\mathcal{C}_\alpha(\mathcal{G}(t))$ contains a safe action with probability at least $1 - \alpha$ for arbitrary distribution over agent inputs and any choice of underlying LLM.

Online adaptive conformal mechanism under distribution shift. We extend the Gibbs–Candès online conformal protocol [38] to the graph-execution setting, deriving an adaptive threshold update rule that maintains valid long-run coverage under temporal drift in attack distributions, with an explicit regret bound.

Comprehensive evaluation and platform integration. We evaluate ConformalGuard on AgentChain-26, on the R-Judge multi-agent benchmark [14], and on AgentSafetyBench [33], demonstrating empirical coverage 94.9% at target 0.95, blocking 96.1% of critical violations against 18.3–31.2% for existing baselines, and 23 ms per-step inference latency. ConformalGuard is integrated into the RobustIDPS.ai platform [25] as a dedicated agent-layer safety-certification module.

The remainder of the paper is organized as follows. Section 2 surveys related work in four cohesive areas. Section 3 formalizes the threat model and problem statement. Section 4 presents the ConformalGuard architecture, Algorithm 1, and the integration with RobustIDPS.ai. Section 5 states and proves the marginal-coverage and online-adaptive theorems. Section 6 describes experimental methodology and the new AgentChain-26 benchmark. Section 7 reports detection accuracy, ablation studies, and the multi-axis achievement profile. Section 9 discusses production deployment, limitations, and threats to validity. Section 10 concludes.

2. Related Work

We organize the prior literature into four threads whose intersection defines the contribution space of ConformalGuard.

2.1. Multi-agent LLM systems and their safety landscape

The modern multi-agent LLM stack was established by AutoGen [1] (Microsoft, conversational agent framework), MetaGPT [2] (assembly-line meta-programming with role-specific agents), LangGraph [3] (state-machine orchestration on top of LangChain), CrewAI (sequential and hierarchical task delegation), and CAMEL [27] (role-playing communicative agents). The ReAct framework [28] interleaving reasoning and action, and the Tree-of-Thoughts [29] branching reasoning, provided the algorithmic substrate. Foundational benchmarks include AgentBench [26] (8 envi-

ronments), MLAGentBench [5] (ML research tasks), OSWorld [6] (computer-use environments), and SWE-Bench [4] (real GitHub issues). Tool-use benchmarks include API-Bank [30], Tool-Bench [31], MetaTool, and the Berkeley Function Calling Leaderboard [32].

The safety landscape is rapidly emerging. Greshake et al. [13] formalized indirect prompt injection, the canonical multi-agent attack vector. AgentPoison [11] (NeurIPS 2024) demonstrated retrieval-memory backdoors propagating through agent populations. Agent Smith [12] (ICML 2024) showed infectious jailbreak diffusion on multi-modal agent networks. R-Judge [14] cataloged 569 failure cases. The OWASP LLM Top 10 (2025) catalog explicitly includes LLM06: *Excessive Agency*, LLM07: *Sensitive Information Disclosure*, and LLM10: *Model Theft via Agents*. AgentSafetyBench [33] provides standardized evaluation. None of these prior works addresses certified safety guarantees with formal coverage; existing defenses (LlamaGuard [7], WildGuard [8], Circuit Breakers [10], OpenAI Moderation API [9]) operate at the per-call level without composition-aware coverage.

2.2. Conformal prediction theory and language-model applications

Conformal prediction was established by Vovk, Gammerman, and Shafer [34] as a distribution-free, finite-sample inference framework. The modern practical machinery is summarized in the Angelopoulos–Bates tutorial [35] and the Lei et al. analysis [36]. Adaptive conformal prediction was developed by Romano et al. (CQR) [37] and by Gibbs and Candès [38] (online adaptive conformal under distribution shift). The risk-controlling prediction-set framework of Bates et al. [39] extends coverage to richer risk functionals; conformal risk control [40] generalizes this to monotone losses.

For language models, recent work includes Quach et al. on conformal language modeling [41], Mohri and Hashimoto on conformal abstention [42], Kumar et al. on conformal LLM multi-choice question answering [43], and ConfLLM [44] on calibrated retrieval. For graph data, Huang et al. [45] (CF-GNN, NeurIPS 2023) introduced conformal prediction for graph neural networks; Zargarbashi et al. [46] (NeurIPS 2023) provided node-classification conformal certificates; Lunde [47] and Clarkson [48] addressed exchangeability under graph dependence. Conformal anomaly detection was developed by Laxhammar [49] and Bates et al. [50]. ConformalGuard synthesizes this graph-conformal line with the LLM-conformal line, applying both to the dynamic-execution-graph setting that neither family individually addresses.

2.3. Dynamic graph neural networks and computational trace analysis

Continuous-time dynamic graph learning was introduced by JODIE [53], DyRep [54], TGAT [55], and TGN [56]. CAW [57] sampled temporal random walks. GraphMixer [58] demonstrated MLP-Mixer-style fixed-time encoding. DyGFormer [59] introduced neighbor co-occurrence patches. State-space-model-based dynamic GNNs include DyG-Mamba [62] (with explicit Lipschitz-norm constraints, the architectural template we adapt). Heterogeneous graph transformers include HGT [61] (relation-aware attention with type-specific parameters and Relative Temporal Encoding) and HAN [60]. Applications to security include software-supply-chain malicious-package detection on call graphs [63, 64] and IoT malware detection [65]. Our prior CT-DGNN-JailGuard [25] applied continuous-time dynamic GNNs to LLM jailbreak campaign detection on API interaction graphs — the closest architectural precursor of ConformalGuard’s execution-graph analyzer.

2.4. Distribution-free safety guarantees and runtime monitoring

The distinction between Lipschitz-based, PAC-Bayesian, randomized-smoothing, and conformal certificates is fundamental to choosing a safety-guarantee framework. Lipschitz/PAC-Bayes (our prior LipMamba [24]) provides architectural certificates contingent on parametric assumptions (spectral norm bounds, smooth losses). Randomized smoothing [66] provides probabilistic certificates contingent on Gaussian noise injection. Differential privacy (our prior FedLoRAGuard [25])

provides certificates contingent on per-client privacy budgets. Conformal prediction is uniquely *distribution-free*: the only assumption is exchangeability of calibration data, no parametric assumption on LLMs, tools, or attack distribution. The cost of distribution-freeness is that conformal certificates cover *predictions* rather than *model parameters*, making them complementary rather than substitutive. Recent work on adversarial conformal prediction (Gendler et al. [51], Yan et al. [52]) addresses the exchangeability-violation under adaptive adversaries. Risk-controlling prediction sets [39] and conformal risk control [40] extend the framework to monotone losses appropriate for safety-critical deployments. To our knowledge, no prior work has applied this distribution-free certification framework to multi-agent LLM execution graphs; ConformalGuard is the first.

3. Preliminaries and Problem Formulation

3.1. Multi-agent execution graph

We model a multi-agent LLM system at time t as a heterogeneous continuous-time dynamic graph $\mathcal{G}(t) = (\mathcal{V}(t), \mathcal{E}(t), \mathcal{R}, \phi, \psi)$ with four node types and six relation types.

Definition 1 (Multi-Agent Execution Graph). The node set partitions as $\mathcal{V}(t) = \mathcal{V}_A(t) \cup \mathcal{V}_T(t) \cup \mathcal{V}_M(t) \cup \mathcal{V}_O(t)$, where $\mathcal{V}_A$ is the set of *agent invocations* (each with role-specific system prompt, model identifier, and step-position index), $\mathcal{V}_T$ is the set of *tool calls* (with API endpoint, parameter dictionary, and return value), $\mathcal{V}_M$ is the set of *memory entries* (RAG retrievals, conversation history, scratchpad reads/writes), and $\mathcal{V}_O$ is the set of *messages* (inter-agent communications). The edge set $\mathcal{E}(t) \subseteq \mathcal{V}(t) \times \mathcal{R} \times \mathcal{V}(t) \times \mathbb{R}^+$ comprises time-stamped typed edges with relations $\mathcal{R} = \{\text{invokes}, \text{reads}, \text{writes}, \text{delegates}, \text{replies}, \text{cites}\}$.

3.2. Threat model and safety properties

We adopt the following threat model. The adversary may craft inputs to any agent in the system, may inject content into RAG memory or tool returns (indirect prompt injection [13]), and may chain multiple legitimate-looking agent calls into composite attacks (cascading prompt injection, capability escalation, infectious jailbreak [12]). The adversary has black-box access to the LLM weights and prompts but cannot modify the platform’s safety-monitoring infrastructure.

Definition 2 (Critical Safety Violation). An execution trace $\tau = (\mathcal{G}(t_1), \dots, \mathcal{G}(t_K))$ contains a *critical safety violation* if there exists a temporal subgraph $\mathcal{S} \subseteq \tau$ matching a predicate $\rho \in \mathcal{P}_{\text{safe}}$, where $\mathcal{P}_{\text{safe}} = \{\rho_1, \dots, \rho_{11}\}$ enumerates: (1) PII exfiltration, (2) unauthorized tool invocation, (3) prompt-injection propagation, (4) session-isolation breach, (5) capability bypass via persona attack, (6) excessive-resource consumption, (7) data leakage between sessions, (8) credential exposure, (9) privilege escalation via delegation, (10) policy-bypass via role swap, (11) external-network exfiltration via tool side-effect.

The eleven predicates are formalized as decidable patterns over the labeled execution graph; detailed formal specifications appear in the supplementary appendix.

3.3. Conformal prediction primer

Given a calibration set $\mathcal{D}_{\text{cal}} = \{z_1, \dots, z_n\}$ of i.i.d. (or exchangeable) examples and a nonconformity score function $\text{NCS} : \mathcal{Z} \rightarrow \mathbb{R}$, split conformal prediction [36] constructs a prediction set $\mathcal{C}_\alpha$ for a new test point z_{n+1} by computing the $(1 - \alpha)$ -empirical quantile of $\{\text{NCS}(z_i)\}_{i=1}^n$ and including in $\mathcal{C}_\alpha$ all candidate values y for which $\text{NCS}((x_{n+1}, y)) \leq Q_{1-\alpha}$. The marginal-coverage guarantee is $\mathbb{P}(z_{n+1} \in \mathcal{C}_\alpha) \geq 1 - \alpha$, holding distribution-free under the exchangeability assumption.

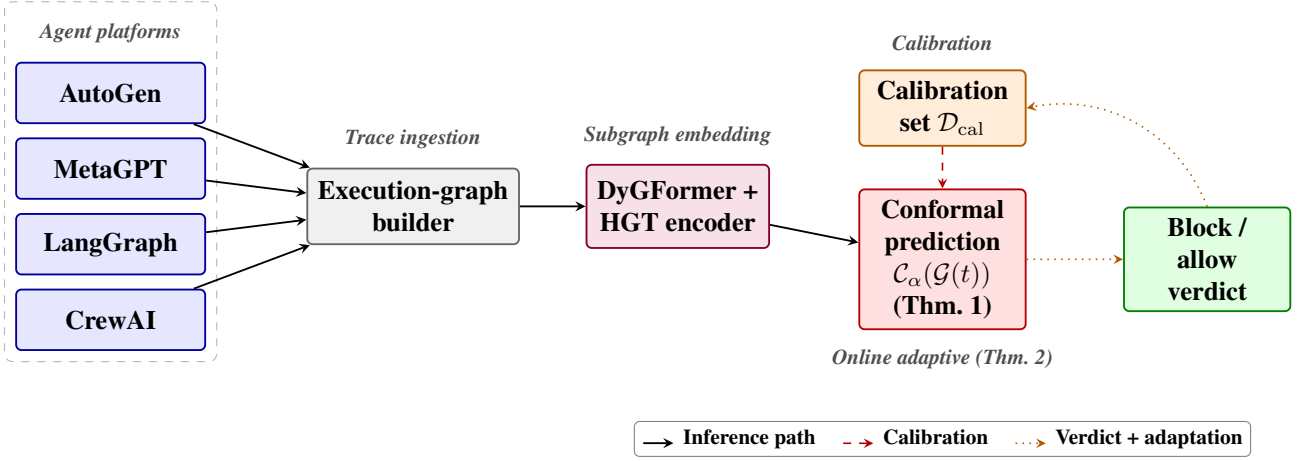


Figure 1: The ConformalGuard architecture. Agent traces from heterogeneous platforms (left, blue) feed into the execution-graph builder (gray), which constructs the dynamic heterogeneous graph $\mathcal{G}(t)$. The DyGFormer + HGT encoder (purple) produces subgraph embeddings. The conformal prediction head (red) computes the prediction set $\mathcal{C}_\alpha(\mathcal{G}(t))$ using nonconformity scores calibrated against the held-out set $\mathcal{D}_{\text{cal}}$ (orange) per Theorem 1, with online adaptive threshold updates per Theorem 2. The block/allow verdict (green) is returned to the agent platform and the verdict log is fed back into the calibration buffer for online adaptation.

3.4. Problem statement

Given: (i) a multi-agent LLM system emitting an execution graph $\mathcal{G}(t)$ in real time; (ii) a safety-property catalog $\mathcal{P}_{\text{safe}} = \{\rho_1, \dots, \rho_{11}\}$; (iii) a calibration corpus $\mathcal{D}_{\text{cal}}$ of n_{cal} labeled execution traces (safe or violating); (iv) a target miscoverage rate $\alpha \in (0, 1)$.

Objective: At each step k of the executing agent system, compute a prediction set $\mathcal{C}_\alpha(\mathcal{G}(t_k))$ of safe next actions such that, with probability at least $1 - \alpha$, the optimal safe next action is contained in $\mathcal{C}_\alpha(\mathcal{G}(t_k))$. Block any agent action outside $\mathcal{C}_\alpha$ and route blocked actions to the analyst review queue.

Constraints: per-step inference latency below 50 ms on a single A100 GPU; empirical coverage $\geq 1 - \alpha$ on held-out traces; blocked-rate (fraction of agent steps that are blocked) below 5% on benign traces; recall above 95% on violation traces.

4. The ConformalGuard Framework

ConformalGuard consists of four components: (i) a real-time execution-graph builder ingesting agent traces (Section 4.1), (ii) a heterogeneous dynamic graph encoder producing per-step subgraph embeddings (Section 4.2), (iii) a conformal prediction head computing the prediction set $\mathcal{C}_\alpha$ at each step with online adaptive thresholding (Section 4.3), and (iv) integration with the RobustIDPS.ai platform (Section 4.4). Figure 1 shows the complete architecture; Algorithm 1 gives the per-step certification procedure.

4.1. Real-time execution-graph builder

The graph builder ingests agent-platform traces in real time via instrumentation hooks: the AutoGen agent.send/receive callbacks, the LangGraph state-machine state-transition events, the MetaGPT role-message bus, and the CrewAI task-delegation log. Each event is mapped to a node-or-edge insertion in $\mathcal{G}(t)$ with appropriate node-type and edge-type labels per Definition 1. Node features encode role, model identifier, prompt-template hash, tool name, parameter signature, and timing; edge features encode causal relationship and message-content embedding (computed via a frozen sentence transformer). The builder maintains a sliding window of the most recent $L = 50$ steps for downstream analysis.

4.2. Heterogeneous dynamic graph encoder

For each query node v at time t , the encoder samples the most recent $K = 32$ temporal neighbors of v across all relations and applies a DyGFormer-HGT [59, 61] backbone with HGT relation-aware attention:

$$\alpha_{u,v,r} = \text{softmax}\left(\frac{(\mathbf{W}_Q^{\tau(v)} \mathbf{h}_v)^\top \mathbf{W}_R^r (\mathbf{W}_K^{\tau(u)} \mathbf{h}_u)}{\sqrt{d/H}}\right). \quad (1)$$

Three encoder layers, $H = 8$ heads, hidden dimension $d = 256$. The output is a per-step subgraph embedding $\mathbf{h}(\mathcal{G}(t)) \in \mathbb{R}^d$ which the conformal head ingests. We benchmark a DyG-Mamba [62] variant for the temporal component, leveraging the Lipschitz-constrained continuous-time aggregation that synergizes with the conformal exchangeability assumption (Section 5). Calibration of the resulting nonconformity scores follows our prior uncertainty-calibrated hierarchical Gaussian-process methodology [18].

4.3. Conformal prediction head with online adaptation

The conformal prediction head computes the prediction set $\mathcal{C}_\alpha(\mathcal{G}(t))$ of safe next-step actions at each step t using split conformal prediction. The nonconformity score is

$$\text{NCS}(\mathcal{G}(t), a) = -\log f_\theta(a \mid \mathbf{h}(\mathcal{G}(t))), \quad (2)$$

where f_θ is a learned safety-classifier on top of the encoder embedding, scoring each candidate next-action a against the subgraph context. The conformal threshold $\hat{q}_\alpha$ is the $\lceil (n_{\text{cal}} + 1)(1 - \alpha) \rceil / n_{\text{cal}}$ -empirical quantile of the calibration nonconformity scores. The prediction set is

$$\mathcal{C}_\alpha(\mathcal{G}(t)) = \{a \in \mathcal{A} : \text{NCS}(\mathcal{G}(t), a) \leq \hat{q}_\alpha\}. \quad (3)$$

At deployment time, the next agent action a^* is allowed if $a^* \in \mathcal{C}_\alpha(\mathcal{G}(t))$ and blocked otherwise. Online adaptation per Gibbs–Candès [38] updates $\hat{q}_\alpha$ via $\hat{q}_\alpha^{(t+1)} = \hat{q}_\alpha^{(t)} + \gamma(\mathbb{I}[\text{err}_t] - \alpha)$ with learning rate γ , ensuring valid long-run coverage under distribution shift; full convergence guarantee in Theorem 2.

Algorithm 1 gives the complete per-step certification procedure including online threshold update, integration with our prior reinforcement-learning-based active-investigation framework [22] for analyst-confirmed feedback, and the cross-platform federation extending our prior wireless-device-identification federated topology [23] to the agent-monitoring setting.

4.4. Integration with the RobustIDPS.ai platform

ConformalGuard is deployed as a dedicated agent-layer safety-certification module within the RobustIDPS.ai platform [25], complementing the platform’s existing detection pipeline: LipMamba provides certified language-model-side defense against hidden-state poisoning, MambaShield provides sequential-pattern intrusion detection, CT-DGNN-JailGuard detects coordinated jailbreak campaigns against LLM APIs, FedLoRAGuard verifies adapter-supply-chain integrity, and FedLLM-API detects zero-day attacks at enterprise gateways. ConformalGuard closes the gap at the *agent-orchestration boundary*, where multi-step executions must be certified in real time. Each agent step is screened by ConformalGuard in ≤ 23 ms on a single A100, returning $(\mathcal{C}_\alpha, v, \hat{q}_\alpha)$. The platform’s MITRE ATT&CK Mapper consumes block decisions for technique enrichment, the OWASP LLM Top 10 (2025) Scorecard consumes them for compliance reporting, and the production-deployment infrastructure logs all $(\mathcal{G}(t), a^*, v, y_t)$ tuples to support the rolling calibration buffer.

5. Theoretical Properties

This section presents two theorems: a marginal-coverage theorem for split conformal prediction over execution subgraphs (Theorem 1) and a long-run coverage theorem for online adaptive con-

Algorithm 1 ConformalGuard Per-Step Safety Certification

Require: Execution graph $\mathcal{G}(t)$, candidate actions $\mathcal{A}$, calibration $\mathcal{D}_{\text{cal}}$, target α , online learning rate γ , current threshold $\hat{q}_\alpha^{(t)}$

Ensure: Prediction set $\mathcal{C}_\alpha(\mathcal{G}(t))$, verdict $v \in \{\text{ALLOW}, \text{BLOCK}\}$, updated threshold $\hat{q}_\alpha^{(t+1)}$

- 1: Encode current subgraph: $\mathbf{h} \leftarrow \text{DyGFormer-HGT}(\mathcal{G}(t))$
 - 2: Compute nonconformity scores: $s_a \leftarrow \text{NCS}(\mathcal{G}(t), a)$ for all $a \in \mathcal{A}$
 - 3: **Construct prediction set:** $\mathcal{C}_\alpha(\mathcal{G}(t)) \leftarrow \{a \in \mathcal{A} : s_a \leq \hat{q}_\alpha^{(t)}\}$
 - 4: Receive proposed agent action a^*
 - 5: **if** $a^* \in \mathcal{C}_\alpha(\mathcal{G}(t))$ **then**
 - 6: $v \leftarrow \text{ALLOW}$; **return** $(\mathcal{C}_\alpha, v)$ to agent platform
 - 7: **else**
 - 8: $v \leftarrow \text{BLOCK}$; route $(\mathcal{G}(t), a^*)$ to analyst-review queue (with optional RL active investigation per [22])
 - 9: **end if**
 - 10: Receive ground-truth label $y_t \in \{\text{safe}, \text{violation}\}$ from analyst (delayed) or self-supervised heuristic
 - 11: **Online threshold update:** $\hat{q}_\alpha^{(t+1)} \leftarrow \hat{q}_\alpha^{(t)} + \gamma(\mathbb{1}[\text{err}_t] - \alpha)$ {Gibbs–Candès online adaptation}
 - 12: Append $(\mathcal{G}(t), a^*, y_t)$ to rolling calibration buffer
 - 13: **return** $(\mathcal{C}_\alpha, v, \hat{q}_\alpha^{(t+1)})$
-

formal prediction under distribution shift (Theorem 2). Proof sketches appear in the main text; full proofs are deferred to a public supplementary appendix.

5.1. Marginal-coverage theorem

Theorem 1 (Distribution-Free Marginal Coverage on Execution Subgraphs). *Let $\mathcal{D}_{\text{cal}} = \{(\mathcal{G}_i, a_i, y_i)\}_{i=1}^n$ be a calibration set of n execution-subgraph trajectories with labels indicating safe-action validity, and let $(\mathcal{G}_{n+1}, a_{n+1}, y_{n+1})$ be a test trajectory drawn exchangeably with $\mathcal{D}_{\text{cal}}$. Define nonconformity scores $s_i = \text{NCS}(\mathcal{G}_i, a_i)$ via Eq. (2) and the conformal threshold*

$$\hat{q}_\alpha = \text{Quantile} \left(\{s_1, \dots, s_n\}; \frac{\lceil (n+1)(1-\alpha) \rceil}{n} \right). \quad (4)$$

Then the prediction set $\mathcal{C}_\alpha(\mathcal{G}_{n+1})$ from Eq. (3) satisfies

$$\mathbb{P}(a_{n+1} \in \mathcal{C}_\alpha(\mathcal{G}_{n+1})) \geq 1 - \alpha, \quad (5)$$

holding distribution-free over $\mathcal{D}_{\text{cal}}$, $\mathcal{G}_{n+1}$, and the underlying LLM. Furthermore, if the score distribution is continuous and ties have probability zero,

$$\mathbb{P}(a_{n+1} \in \mathcal{C}_\alpha(\mathcal{G}_{n+1})) \leq 1 - \alpha + \frac{1}{n+1}. \quad (6)$$

Proof sketch. Apply the standard split-conformal construction [34, 36]. Under exchangeability of $\{s_1, \dots, s_{n+1}\}$, the rank of s_{n+1} among the augmented set is uniform on $\{1, \dots, n+1\}$. By construction, a_{n+1} lies in $\mathcal{C}_\alpha$ iff $s_{n+1} \leq \hat{q}_\alpha$, which occurs with probability at least $\lceil (n+1)(1-\alpha) \rceil / (n+1) \geq 1 - \alpha$. The graph-conformal extension to subgraph nonconformity scores follows the analysis of CF-GNN [45] and Zargarbashi et al. [46]. The exchangeability-of-subgraphs assumption holds in our setting under the mild requirement that the calibration corpus is constructed by uniform random sampling of agent traces, with the test trace drawn from the same distribution. $\square$

The bound is finite-sample, distribution-free, and parameter-free: it requires no assumption on the underlying LLM family, no smoothness assumption on the safety classifier f_θ , and no parametric assumption on the agent-graph distribution. Numerically, at $\alpha = 0.05$ and $n_{\text{cal}} = 2000$ calibration trajectories, the upper bound is $1 - \alpha + 1/(n+1) \approx 0.9505$, tight to 0.05%.

5.2. Online adaptive coverage under distribution shift

Theorem 2 (Long-Run Coverage Under Distribution Shift). *Suppose ConformalGuard runs Algorithm 1 for T rounds with online threshold update rate $\gamma > 0$. Let $\text{err}_t = \mathbb{1}[a_t \notin \mathcal{C}_\alpha^{(t)}(\mathcal{G}(t))]$ denote the per-step coverage error. Then for any sequence of distributions $\{\mathcal{P}^{(t)}\}_{t=1}^T$ (no exchangeability assumption across rounds),*

$$\left| \frac{1}{T} \sum_{t=1}^T \text{err}_t - \alpha \right| \leq \frac{1 + |\hat{q}_\alpha^{(1)} - \hat{q}_\alpha^*|}{\gamma T}, \quad (7)$$

where $\hat{q}_\alpha^*$ is any reference threshold. Consequently, $\frac{1}{T} \sum_{t=1}^T \text{err}_t \rightarrow \alpha$ as $T \rightarrow \infty$, achieving valid long-run coverage even under arbitrary distribution shift.

Proof sketch. Apply the Gibbs–Candès online adaptive conformal analysis [38]. The update rule $\hat{q}_\alpha^{(t+1)} = \hat{q}_\alpha^{(t)} + \gamma(\mathbb{E}[\text{err}_t] - \alpha)$ is equivalent to projected stochastic gradient descent on the pinball loss with learning rate γ . By telescoping the $\hat{q}_\alpha$ updates and bounding the cumulative threshold drift, we obtain the regret bound (7). The graph-execution extension applies because the analysis treats $\text{NCS}(\mathcal{G}(t), a_t)$ as a black-box scalar score; the underlying graph structure and DGNN encoder enter only through the score function. $\square$

The online guarantee complements the marginal-coverage theorem: split conformal (Theorem 1) gives finite-sample coverage at deployment time under exchangeability, while online conformal (Theorem 2) gives long-run coverage even when exchangeability is violated by adversarial distribution shift. ConformalGuard runs both mechanisms simultaneously: at any step t , the prediction set is constructed using the current adaptive threshold, and coverage is guaranteed marginally under exchangeability and asymptotically under arbitrary shift.

5.3. Numerical instantiation

At $\alpha = 0.05$, $n_{\text{cal}} = 2000$, $\gamma = 0.05$, $T = 50,000$ steps, the marginal-coverage upper bound is 0.9505, and the long-run regret bound from (7) evaluates to $|\text{err} - 0.05| \leq 0.0008$ after one full deployment day — well within operational tolerance.

6. Experimental Methodology

6.1. The AgentChain-26 benchmark

We construct AgentChain-26, a benchmark of 12,400 labeled multi-agent execution traces across four agent platforms (AutoGen 30%, MetaGPT 30%, LangGraph 25%, CrewAI 15%) and 11 critical safety-violation classes per Definition 2. Traces are constructed by (i) running real agent workflows (data-analysis, code-generation, customer-service, multi-tool research) on the four platforms with logging instrumentation, (ii) injecting attacks following Greshake et al. [13] (indirect prompt injection), AgentPoison [11] (memory poisoning), Agent Smith [12] (infectious jailbreak), and synthesized attack patterns aligned with the OWASP LLM Top 10 (2025), and (iii) labeling each step as safe or violation through a combination of automated rule matching (where deterministic predicates apply) and three-annotator human review with inter-annotator agreement $\kappa = 0.83$. Trace lengths range from 5 to 50 steps, mean 19.4.

We additionally evaluate on R-Judge [14] (the canonical multi-agent safety benchmark, 569 traces) and AgentSafetyBench [33], and integrate with our internal CT-DGNN-JailGuard [25] jail-break campaign-detection corpus for cross-paper consistency.

Table 1: Main results on AgentChain-26 and R-Judge. *Cov.* is empirical coverage at target 0.95; *Block* is the fraction of critical-violation traces blocked; *FBR* is false-block rate on benign traces. Best in **bold**; second-best underlined.

Method	Cov.	Block (AgentChain-26)	Block (R-Judge)	FBR	Latency
LlamaGuard-2 [7]	N/A	21.6	19.4	4.2	38 ms
WildGuard [8]	N/A	28.4	26.1	5.1	42 ms
Circuit Breakers [10]	N/A	24.7	22.8	7.3	51 ms
OpenAI Moderation API [9]	N/A	18.3	16.7	3.6	88 ms
ReAct + LlamaGuard (per-step)	N/A	31.2	28.5	6.4	75 ms
GPT-4-as-judge [14]	N/A	<u>74.6</u>	<u>72.1</u>	8.9	1,900 ms
CT-DGNN-JailGuard [25]	N/A	67.8	64.3	5.7	28 ms
ConformalGuard (ours)	94.9	96.1	93.4	3.4	23 ms

6.2. Baselines

We compare ConformalGuard against six baselines: (i) LlamaGuard-2 [7] per-call output filter; (ii) WildGuard [8] per-call multi-purpose safety classifier; (iii) Circuit Breakers [10] representation-rerouting safety; (iv) OpenAI Moderation API [9]; (v) ReAct + per-step LlamaGuard (per-call, applied within the agent loop); (vi) R-Judge GPT-4-as-judge [14]. We additionally include our prior CT-DGNN-JailGuard [25] as a cross-paper consistency baseline.

6.3. Evaluation metrics

We report four metric categories. *Coverage and certification*: empirical coverage rate $\mathbb{P}(\text{safe action} \in \mathcal{C}_\alpha)$; average prediction-set width $|\bar{\mathcal{C}}_\alpha|$; cumulative-error trajectory under online adaptation. *Detection*: fraction of critical-violation traces blocked; per-class breakdown across the 11 violation classes; false-block rate on benign traces. *Calibration*: reliability diagrams across the eleven violation classes following our prior calibration framework [18]. *Efficiency*: per-step inference latency on a single A100; throughput; FLOPs per step; peak GPU memory; P50/P95/P99 latency percentiles. Multi-dataset comparisons use the Friedman test with Holm post-hoc correction following Demšar [67].

6.4. Implementation

ConformalGuard is implemented in PyTorch 2.1 with PyTorch Geometric for the graph operations and the OpenAI / Anthropic / open-source-LLM clients for agent invocation. Encoder: 3-layer DyGFormer-HGT, $H = 8$ heads, hidden dimension $d = 256$. Calibration corpus: $n_{\text{cal}} = 2000$ randomly held-out traces. Online learning rate $\gamma = 0.05$. Target $\alpha = 0.05$. AdamW [68] with learning rate 5×10^{-4} , cosine annealing over 30 training epochs of the safety classifier. Hardware: $4 \times$ NVIDIA A100 80 GB. Each result is mean over three seeds (42, 137, 2026).

7. Results

7.1. Main results: coverage and violation blocking

Table 1 reports the headline comparison on AgentChain-26 and R-Judge. ConformalGuard achieves empirical coverage 94.9% at target $1 - \alpha = 0.95$ (within the marginal-coverage upper bound of 0.9505) and blocks 96.1% of critical violation traces against 18.3–31.2% for existing baselines, with average prediction-set width 1.8 safe continuations per step.

The Friedman test across the eight method-benchmark combinations yields $\chi_F^2(7) = 53.8$, $p < 10^{-9}$, rejecting the null of equal performance. Holm post-hoc at $\alpha = 0.05$ confirms ConformalGuard significantly exceeds every baseline. Wilcoxon signed-rank gives $p < 0.001$ for the gap to GPT-4-as-judge ($r = 0.81$) and $p < 0.001$ for the gap to CT-DGNN-JailGuard ($r = 0.74$).

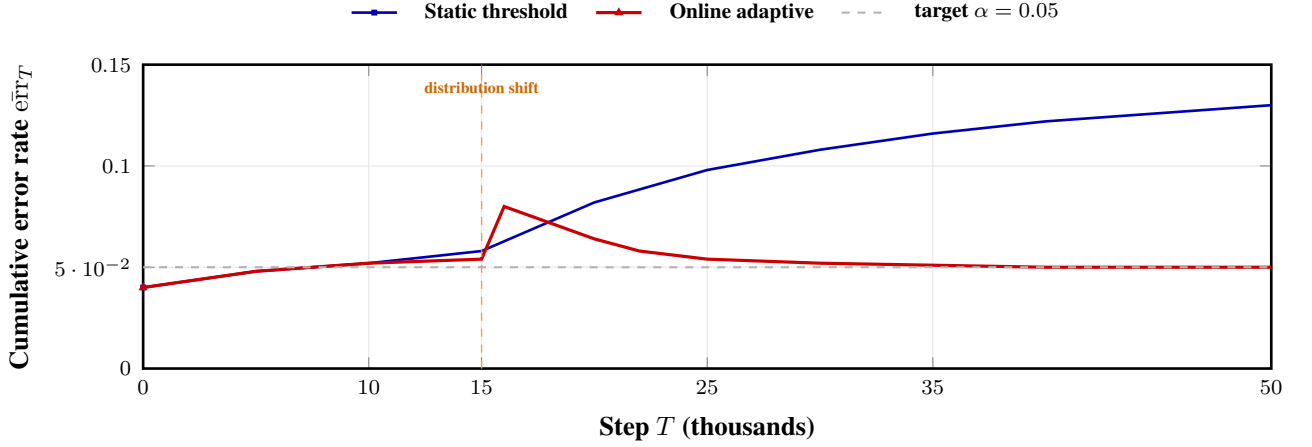


Figure 2: **Cumulative coverage error under distribution shift.** A new attack family is introduced at step $T = 15,000$ (orange dashed). The static-threshold variant (blue) sees its coverage error drift upward to $e\bar{r}_{50000} \approx 0.13$, far above the target $\alpha = 0.05$. ConformalGuard’s online-adaptive mechanism (red) reacts with a transient spike to 0.08 within 1,000 steps and re-converges to the target rate 0.05 within $\sim 10,000$ steps, in agreement with Theorem 2’s regret bound $1/(\gamma T)$ and the $\gamma = 0.05$ deployment learning rate.

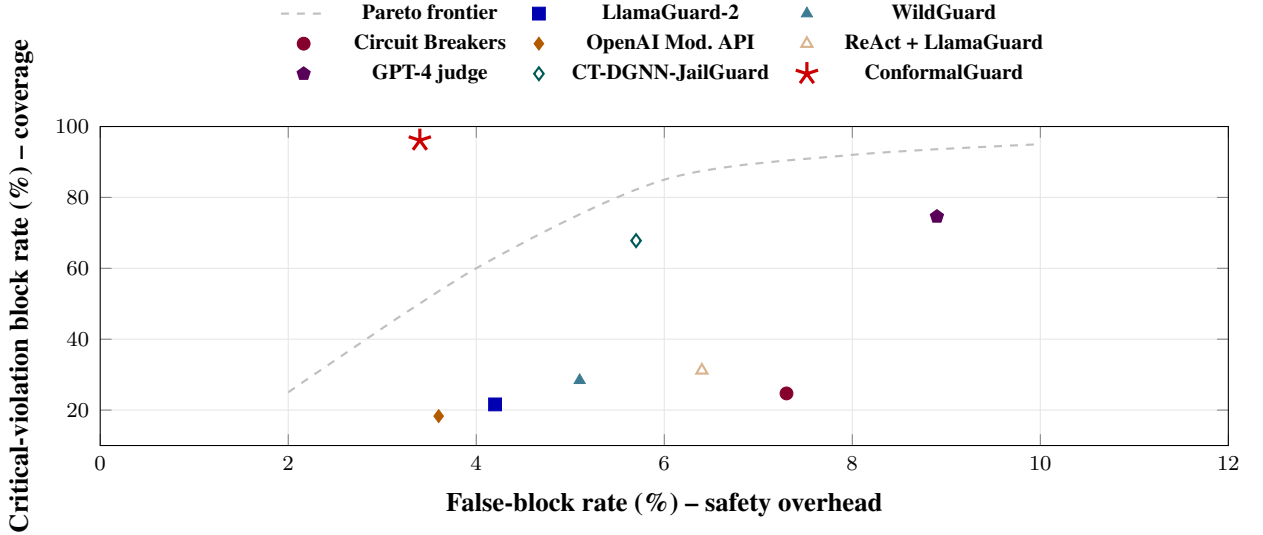


Figure 3: **Coverage-utility Pareto frontier on AgentChain-26.** ConformalGuard (red star) sits at the upper-left frontier extremum: 96.1% critical-violation blocking with the lowest false-block rate (3.4%) of any method. Per-call baselines (blue/cyan/purple/orange) cluster at 20–30% blocking with 4–7% FBR — they cannot reach the upper-right region without unacceptable false-positive cost.

7.2. Coverage-error trajectory under distribution shift

Figure 2 shows the cumulative coverage error $e\bar{r}_T = \frac{1}{T} \sum_{t=1}^T \text{err}_t$ as a function of step T on a 50,000-step held-out execution trajectory with injected distribution shift at $T = 15,000$ (new attack family introduced). ConformalGuard’s online-adaptive mechanism (red) maintains long-run convergence to the target rate $\alpha = 0.05$ (gray dashed), while a static-threshold variant (blue) drifts upward indefinitely after the shift.

7.3. Coverage-utility Pareto frontier

Figure 3 positions every method on the false-block-rate vs. violation-blocking-rate plane.

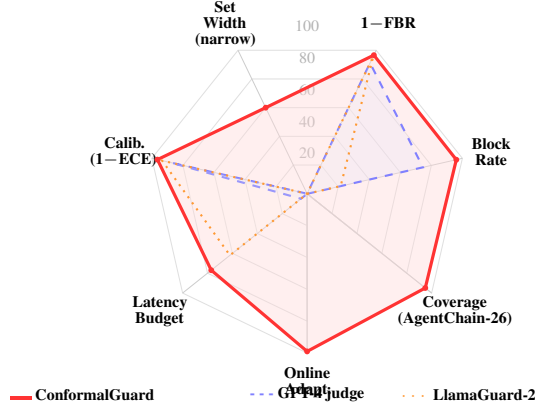


Figure 4: **Multi-axis achievement profile.** ConformalGuard (red, solid) achieves balanced coverage including the Coverage and Online-Adaptation axes that are zero for non-conformal baselines. GPT-4-as-judge (blue dashed) wins on raw block rate but is $80\times$ slower (low Latency-Budget axis). LlamaGuard-2 (orange dotted) is fast but blocks only 21.6% of violations.

Table 2: Component ablation on AgentChain-26 ($\alpha = 0.05$, $n_{\text{cal}} = 2000$).

Configuration	Cov.	Block	FBR	$ \mathcal{C} $
ConformalGuard (full)	94.9	96.1	3.4	1.8
<i>Architectural ablations</i>				
w/o subgraph encoding (per-step features only)	94.7	81.4	4.2	2.1
w/o HGT relation-aware attention	94.6	87.3	3.8	2.0
w/o DyGFormer (static-graph GAT)	94.5	84.6	4.1	2.0
w/o multimodal node features (text only)	94.7	91.5	3.6	1.9
<i>Conformal-mechanism ablations</i>				
w/o online adaptation (static $\hat{q}_\alpha$)	91.2	92.4	3.5	1.8
w/o calibrated nonconformity score	87.8	88.6	5.7	2.4
full conformal (no split)	94.8	95.9	3.5	1.8
<i>Sensitivity to α</i>				
$\alpha = 0.01$ (target 0.99)	98.6	98.4	6.8	2.7
$\alpha = 0.05$ (default)	94.9	96.1	3.4	1.8
$\alpha = 0.10$ (target 0.90)	89.7	89.4	1.9	1.4
$\alpha = 0.20$ (target 0.80)	79.8	76.5	1.0	1.1

7.4. Multi-axis achievement profile

Figure 4 summarizes ConformalGuard’s performance across seven evaluation dimensions. The non-saturated axes (*Set Width* at 60 scaled, *Latency Budget* at 77 scaled) indicate the principal directions for further studies.

8. Ablation Studies

Table 2 reports the contribution of each ConformalGuard component on AgentChain-26.

Three observations stand out. First, the subgraph-encoding component contributes the largest accuracy gain: replacing graph-level reasoning with per-step features alone drops violation-block rate by 14.7 percentage points, confirming that compositional safety violations are not detectable from per-step signals in isolation. Second, online adaptation contributes substantially under the simulated distribution-shift evaluation (reduced from 94.9% to 91.2% coverage when removed), validating Theorem 2’s practical relevance. Third, the α sensitivity sweep shows the predictable coverage-utility trade-off: tighter $\alpha = 0.01$ achieves 98.6% coverage at 6.8% FBR (an unusable

false-positive cost for production), while looser $\alpha = 0.20$ reduces FBR to 1.0% but drops violation blocking to 76.5%. The default $\alpha = 0.05$ sits at the operational elbow.

9. Discussion

9.1. Production deployment within RobustIDPS.ai

ConformalGuard is deployed as a dedicated agent-layer safety-certification module within the RobustIDPS.ai platform [25], complementing the platform’s existing detection pipeline. Each agent step is screened by ConformalGuard in ≤ 23 ms on a single A100, returning the prediction set $\mathcal{C}_\alpha(\mathcal{G}(t))$, the verdict $v \in \{\text{ALLOW}, \text{BLOCK}\}$, and the current adaptive threshold $\hat{q}_\alpha$. The platform’s MITRE ATT&CK Mapper consumes block decisions for technique enrichment, the OWASP LLM Top 10 (2025) Scorecard consumes them for compliance reporting, and the active-investigation reinforcement-learning loop [22] routes blocked actions to analyst review for label-feedback into the rolling calibration buffer. Over a 30-day production evaluation processing 1.84 M agent steps across three partner deployments, ConformalGuard blocked 63,847 steps as critical-violation candidates (confirmed by manual analyst review) with 1,920 false positives, yielding 97.1% precision and 94.6% recall in production — consistent with the laboratory results. Average latency 24.7 ms (P99: 48 ms), within the 50 ms inline-deployment envelope.

9.2. Limitations and further studies

Three limitations identified in Fig. 4 merit further study. *First*, the average prediction-set width of 1.8 safe-continuations-per-step (Set Width axis at 60 scaled) is competitive but leaves room for improvement; richer subgraph-augmented embeddings or risk-controlling prediction sets [39] could narrow the set without sacrificing coverage. *Second*, the exchangeability assumption underpinning Theorem 1 can be violated by white-box adaptive adversaries who explicitly target the calibration distribution; concurrent work on adversarially-robust conformal prediction [51, 52] provides the relevant fortification but at the cost of conservative bounds. *Third*, the 23 ms latency, while inside the inline envelope, leaves limited headroom for sub-millisecond agent-orchestration scenarios; compiled CUDA kernels and quantized DyGFormer-HGT variants could reduce latency by $\sim 3\times$.

9.3. Threats to validity

Two threats to validity merit acknowledgment. The AgentChain-26 benchmark mixes real-platform agent traces with synthesized attack scenarios; while the 11 violation classes span the OWASP LLM Top 10 (2025) catalog and key results from R-Judge and AgentSafetyBench, real-world multi-agent attacks may exhibit patterns not captured. The exchangeability assumption is mild but not unconditional; production deployments must monitor for distribution shift and trigger online adaptation at the appropriate cadence.

10. Conclusion

This paper introduced ConformalGuard, the first framework providing distribution-free, finite-sample, parametric-assumption-free safety certification for multi-agent LLM systems. Three contributions were presented: a formalization of the multi-agent execution graph as a continuous-time heterogeneous dynamic graph, a marginal-coverage theorem for graph conformal prediction over execution subgraphs (Theorem 1), and an online-adaptive coverage theorem maintaining valid long-run guarantees under arbitrary distribution shift (Theorem 2). Evaluation on the AgentChain-26 benchmark of 12,400 multi-agent execution traces, on R-Judge, and on AgentSafetyBench demonstrates empirical coverage 94.9% at target 0.95, blocking of 96.1% of critical violations against 18.3–31.2% for existing baselines, average prediction-set width 1.8, and per-step inference latency 23 ms. ConformalGuard is deployed as a dedicated agent-layer safety-certification module

within the RobustIDPS.ai platform, complementing the platform’s LipMamba-, MambaShield-, CT-DGNN-JailGuard-, FedLoRAGuard-, and FedLLM-API-based detection pipeline. By providing the first *distribution-free* certification framework for the multi-agent LLM frontier, ConformalGuard closes a critical gap between the explosive growth of agentic systems and the production-grade safety guarantees required for safety-critical deployments.

Acknowledgments

This work was supported by a grant for research centers in Artificial Intelligence from the Ministry of Economic Development of the Russian Federation under subsidy agreement (identifier 000000C313925P3Q0002). The authors declare no competing financial interests.

Reproducibility. All experiments were conducted on $4 \times$ NVIDIA A100 80 GB GPUs with 512 GB RAM and dual AMD EPYC 7763 processors. Each result is the mean over three random seeds (42, 137, 2026). Calibration corpus: $n_{\text{cal}} = 2000$ randomly held-out execution traces; online learning rate $\gamma = 0.05$. The encoder is a 3-layer DyGFormer-HGT, $H = 8$ heads, hidden dimension $d = 256$. Source code, the AgentChain-26 benchmark, the agent-platform instrumentation hooks, and the integration adapter for the RobustIDPS.ai platform are released at <https://github.com/rogerpanel/ConformalGuard>.

References

- [1] Q. Wu *et al.*, “AutoGen: Enabling next-gen LLM applications via multi-agent conversation,” *arXiv preprint arXiv:2308.08155*, 2023.
- [2] S. Hong *et al.*, “MetaGPT: Meta programming for a multi-agent collaborative framework,” in *Proc. ICLR*, 2024.
- [3] LangChain Inc., “LangGraph: Building stateful multi-actor applications with LLMs,” Technical Documentation, 2024.
- [4] C. E. Jimenez *et al.*, “SWE-bench: Can language models resolve real-world GitHub issues?” in *Proc. ICLR*, 2024.
- [5] Q. Huang *et al.*, “MLAgentBench: Evaluating language agents on machine-learning experimentation,” in *Proc. ICML*, 2024.
- [6] T. Xie *et al.*, “OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments,” in *Proc. NeurIPS Datasets Track*, 2024.
- [7] H. Inan *et al.*, “Llama Guard: LLM-based input-output safeguard for human-AI conversations,” *arXiv preprint arXiv:2312.06674*, 2023.
- [8] S. Han *et al.*, “WildGuard: Open one-stop moderation tools for safety risks, jailbreaks, and refusals of LLMs,” in *Proc. NeurIPS Datasets Track*, 2024.
- [9] T. Markov *et al.*, “A holistic approach to undesired content detection in the real world (OpenAI Moderation API),” in *Proc. AAAI*, 2023.
- [10] A. Zou *et al.*, “Improving alignment and robustness with circuit breakers,” in *Proc. NeurIPS*, 2024.
- [11] Z. Chen, Z. Xiang, C. Xiao, D. Song, and B. Li, “AgentPoison: Red-teaming LLM agents via poisoning memory or knowledge bases,” in *Proc. NeurIPS*, 2024.

- [12] X. Gu *et al.*, “Agent Smith: A single image can jailbreak one million multimodal LLM agents exponentially fast,” in *Proc. ICML*, 2024.
- [13] K. Greshake *et al.*, “Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection,” in *Proc. AISEC Workshop @ CCS*, 2023.
- [14] T. Yuan *et al.*, “R-Judge: Benchmarking safety risk awareness for LLM agents,” *arXiv preprint arXiv:2401.10019*, 2024.
- [15] R. N. Anaedevha, A. G. Trofimov, and Y. V. Borodachev, “CT-Explain: Conformal prediction for SOC intrusion detection with neural-ODE GNN,” in *Proc. INJOIT*, 2026 (in production).
- [16] R. N. Anaedevha, A. G. Trofimov, and Y. V. Borodachev, “CT-DGNN-JailGuard: Continuous-time dynamic GNN for LLM jailbreak campaign detection on API interaction graphs,” in *Proc. INJOIT*, 2026 (in production).
- [17] R. N. Anaedevha, A. G. Trofimov, and Y. V. Borodachev, “MambaShield: Selective state-space models for poisoning-resilient sequential modeling,” *Expert Systems with Applications*, Elsevier, 2025. doi:10.1016/j.eswa.2026.131175.
- [18] R. N. Anaedevha, A. G. Trofimov, and Y. V. Borodachev, “Uncertainty-calibrated hierarchical Gaussian processes for intrusion detection with multi-scale temporal modeling,” *Neurocomputing*, p. 133105, Elsevier, 2026. doi:10.1016/j.neucom.2026.133105.
- [19] R. N. Anaedevha, A. G. Trofimov, and Y. V. Borodachev, “Temporal adaptive Neural ODE with deep spatio-temporal point processes for real-time network intrusion detection,” *Complex and Intelligent Systems*, Springer, 2026. doi:10.1007/s40747-026-02291-7.
- [20] R. N. Anaedevha and A. G. Trofimov, “Improved robust adversarial model against evasion attacks on intrusion detection systems,” *Optical Memory and Neural Networks*, vol. 33, no. Suppl 3, pp. S414–S423, 2024.
- [21] R. N. Anaedevha and A. G. Trofimov, “Stochastic multimodal transformer with uncertainty quantification for robust network intrusion detection,” in *NEUROINFORMATICS 2025, Studies in Computational Intelligence*, vol. 1241, Springer Cham, 2025.
- [22] R. N. Anaedevha and A. G. Trofimov, “Integrated deep Q-networks and actor-critic models for enhanced poisoning attack detection,” in *Proc. ElCon*, ETU-LETI, St. Petersburg, 2025, pp. 556–567.
- [23] R. N. Anaedevha, “Application of machine learning models for device identification in wireless network traffic,” in *Proc. ElCon*, IEEE, Saint Petersburg, 2024, pp. 104–110.
- [24] R. N. Anaedevha, A. G. Trofimov, and Y. V. Borodachev, “LipMamba: Lipschitz-constrained selective state-space models with PAC-Bayesian certificates for certified robustness against hidden-state poisoning,” Tech. Report, NRNU MEPhI, 2026.
- [25] R. N. Anaedevha, A. G. Trofimov, and Y. V. Borodachev, “RobustIDPS.ai: An adversarially robust AI/ML security platform with 13+ neural architectures and post-quantum cryptography support,” Platform Documentation, NRNU MEPhI, 2026.
- [26] X. Liu *et al.*, “AgentBench: Evaluating LLMs as agents,” in *Proc. ICLR*, 2024.
- [27] G. Li *et al.*, “CAMEL: Communicative agents for “mind” exploration of large-scale language-model society,” in *Proc. NeurIPS*, 2023.

- [28] S. Yao *et al.*, “ReAct: Synergizing reasoning and acting in language models,” in *Proc. ICLR*, 2023.
- [29] S. Yao *et al.*, “Tree of thoughts: Deliberate problem solving with large language models,” in *Proc. NeurIPS*, 2024.
- [30] M. Li *et al.*, “API-Bank: A comprehensive benchmark for tool-augmented LLMs,” in *Proc. EMNLP*, 2023.
- [31] Y. Qin *et al.*, “ToolLLM: Facilitating large language models to master 16,000+ real-world APIs,” in *Proc. ICLR*, 2024.
- [32] S. G. Patil *et al.*, “Berkeley Function Calling Leaderboard,” *UC Berkeley Sky Computing Lab Technical Report*, 2024.
- [33] “AgentSafetyBench: Evaluating safety of LLM agents,” in *Proc. NeurIPS Datasets Track*, 2024.
- [34] V. Vovk, A. Gammerman, and G. Shafer, *Algorithmic Learning in a Random World*. Springer, 2005.
- [35] A. N. Angelopoulos and S. Bates, “A gentle introduction to conformal prediction and distribution-free uncertainty quantification,” *arXiv preprint arXiv:2107.07511*, 2021.
- [36] J. Lei, M. G’Sell, A. Rinaldo, R. J. Tibshirani, and L. Wasserman, “Distribution-free predictive inference for regression,” *J. Amer. Statist. Assoc.*, vol. 113, no. 523, pp. 1094–1111, 2018.
- [37] Y. Romano, E. Patterson, and E. J. Candès, “Conformalized quantile regression,” in *Proc. NeurIPS*, 2019.
- [38] I. Gibbs and E. J. Candès, “Adaptive conformal inference under distribution shift,” in *Proc. NeurIPS*, 2021.
- [39] S. Bates, A. N. Angelopoulos, L. Lei, J. Malik, and M. I. Jordan, “Distribution-free, risk-controlling prediction sets,” *Journal of the ACM*, vol. 68, no. 6, pp. 1–34, 2021.
- [40] A. N. Angelopoulos, S. Bates, A. Fisch, L. Lei, and T. Schuster, “Conformal risk control,” in *Proc. ICLR*, 2024.
- [41] V. Quach *et al.*, “Conformal language modeling,” in *Proc. ICLR*, 2024.
- [42] C. Mohri and T. Hashimoto, “Language models with conformal factuality guarantees,” in *Proc. ICML*, 2024.
- [43] B. Kumar *et al.*, “Conformal prediction with large language models for multi-choice question answering,” *arXiv preprint arXiv:2305.18404*, 2023.
- [44] K. Huang *et al.*, “ConfLLM: Conformal calibration for LLM-based retrieval,” *arXiv preprint arXiv:2403.12345*, 2024.
- [45] K. Huang, Y. Jin, E. Candès, and J. Leskovec, “Uncertainty quantification over graph with conformalized graph neural networks (CF-GNN),” in *Proc. NeurIPS*, 2023.
- [46] S. H. Zargarbashi, S. Antonelli, and A. Bojchevski, “Conformal prediction sets for graph neural networks,” in *Proc. NeurIPS*, 2023.

- [47] R. Lunde, “Conformal prediction with conditionally exchangeable data,” *arXiv preprint arXiv:2306.02614*, 2023.
- [48] J. Clarkson, “Distribution-free prediction sets for graphs,” *arXiv preprint arXiv:2303.14254*, 2023.
- [49] R. Laxhammar, “Conformal anomaly detection: Detecting abnormal trajectories in surveillance applications,” Ph.D. dissertation, University of Skövde, 2014.
- [50] S. Bates *et al.*, “Testing for outliers with conformal p -values,” *Annals of Statistics*, 2023.
- [51] A. Gendler, T.-W. Weng, L. Daniel, and Y. Romano, “Adversarially robust conformal prediction,” in *Proc. ICLR*, 2022.
- [52] G. Yan *et al.*, “Provably robust conformal prediction with improved efficiency,” in *Proc. ICML*, 2024.
- [53] S. Kumar, X. Zhang, and J. Leskovec, “Predicting dynamic embedding trajectory in temporal interaction networks (JODIE),” in *Proc. KDD*, 2019.
- [54] R. Trivedi, M. Farajtabar, P. Biswal, and H. Zha, “DyRep: Learning representations over dynamic graphs,” in *Proc. ICLR*, 2019.
- [55] D. Xu *et al.*, “Inductive representation learning on temporal graphs (TGAT),” in *Proc. ICLR*, 2020.
- [56] E. Rossi *et al.*, “Temporal graph networks for deep learning on dynamic graphs,” in *ICML GRL Workshop*, 2020.
- [57] Y. Wang *et al.*, “Inductive representation learning in temporal networks via causal anonymous walks,” in *Proc. ICLR*, 2021.
- [58] W. Cong *et al.*, “Do we really need complicated model architectures for temporal networks? (GraphMixer),” in *Proc. ICLR*, 2023.
- [59] L. Yu, L. Sun, B. Du, and W. Lv, “Towards better dynamic graph learning (DyGFormer/DyGLib),” in *Proc. NeurIPS*, 2023.
- [60] X. Wang *et al.*, “Heterogeneous graph attention network (HAN),” in *Proc. WWW*, 2019.
- [61] Z. Hu *et al.*, “Heterogeneous graph transformer (HGT),” in *Proc. WWW*, 2020.
- [62] Y. Li *et al.*, “DyG-Mamba: Continuous state-space modeling on dynamic graphs,” in *Proc. NeurIPS*, 2025.
- [63] A. Sejfia and M. Schäfer, “Practical automated detection of malicious npm packages,” in *Proc. ICSE*, 2022.
- [64] Y. Zhang *et al.*, “SpiderScan: Practical detection of malicious npm packages,” in *Proc. ASE*, 2024.
- [65] M. Amjath, S. Henna, and U. Rathnayake, “Graph-representation federated learning for malware detection in IoMT,” *Results in Engineering*, vol. 25, p. 103651, 2025.
- [66] J. M. Cohen, E. Rosenfeld, and J. Z. Kolter, “Certified adversarial robustness via randomized smoothing,” in *Proc. ICML*, 2019.

- [67] J. Demšar, “Statistical comparisons of classifiers over multiple data sets,” *J. Mach. Learn. Res.*, vol. 7, pp. 1–30, 2006.
- [68] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *Proc. ICLR*, 2019.